

# RISC-V 环境下 llama.cpp 大模型部署与性能测试调研报告

项目 内容

课题来源 上海苦芽科技有限公司暑期实习

调研方向 开放指令集架构 RISC-V × 大语言模型推理

作者 王曜之

日期 2026 年 8 月

## 摘要

本课题围绕“开放指令集架构 **RISC-V** 能否承载大语言模型（LLM）推理”这一核心问题，完成了从运行环境搭建、推理框架移植、量化模型部署到推理验证与性能测试的**全链路实验**。实验以 QEMU 模拟 RISC-V 平台并运行 openEuler 24.03 LTS-SP3（RISC-V），交叉编译移植 **llama.cpp**，部署 **Qwen2.5 系列 GGUF 量化模型**（0.5B / 1.5B），完成中文问答、知识解释、代码生成与数学推理等功能的验证，并进一步开启 **RISC-V Vector（RVV）向量扩展**重新编译复测。

主要结果：Qwen2.5-0.5B-Instruct 在纯软件模拟环境下 Prompt 处理约 **3.5 token/s**、生成约 **2.1 token/s**，达到“轻量可用”水平；Qwen2.5-1.5B-Instruct 约为 **0.6 / 0.5 token/s**，受模拟器限制明显。开启 RVV 后 0.5B 模型复测为 **3.3 / 1.8 token/s**，与基线基本持平——说明在 QEMU 模拟器下 RVV 的加速收益被指令模拟开销抵消，真实加速效果有待硬件验证。

结论：轻量化大语言模型在 RISC-V 软件生态中具备**部署与推理的可行性**，环境、编译、部署、推理整条链路均已打通。

## 1 背景与动机

### 1.1 行业趋势：大模型推理走向边缘侧

大语言模型能力快速提升的同时，AI 计算正在从云端向端侧、边缘侧扩展，端侧 / 边缘推理需求持续增长。模型本身不断变小、变轻，这为传统 x86 / ARM 之外的处理器架构带来了新的机会窗口。

### 1.2 RISC-V 的三大特性

- 开放性**：开放指令集架构，任何组织与个人均可使用、扩展；
- 可扩展性**：支持按需定制指令扩展；
- 灵活定制**：可按场景灵活设计处理器。

这些特性天然契合 AI 边缘化场景下的差异化探索需求。

### 1.3 待回答的问题

不满足于纸面讨论，本实验用真实环境回答一个具体问题：**大语言模型在 RISC-V 平台上到底能不能部署、能不能推理、性能如何？**

---

## 2 技术背景

### 2.1 llama.cpp

llama.cpp 是一个以 C/C++ 实现、聚焦本地推理的 LLM 推理框架，依赖少、易于跨平台与交叉编译，支持包括 RISC-V 在内的多种目标架构。其核心产物 `llama-cli` 用于命令行推理测试，`llama-server` 用于提供推理服务。

### 2.2 GGUF 与 Q4\_K\_M 量化

- **GGUF** 是 llama.cpp 原生支持的模型存储格式，将模型权重与元数据封装为单一文件，便于分发与加载；
- **Q4\_K\_M** 属于 4-bit K-quant 量化，在模型体积、内存占用与推理质量之间取得良好平衡，特别适合资源受限的模拟环境。

### 2.3 关键指令集 / ABI 要素

- **rv64gc**: RV64 通用指令集，是基础兼容的保证；
- **zba / zbb / zbs**: 位操作扩展，推理内核（GGML）执行时可受益；
- **v (RVV)**: RISC-V Vector 向量扩展，可对矩阵 / 张量运算做向量化；
- **lp64d**: LP64 数据模型 + 双精度浮点 ABI，浮点运算依赖此项。

---

## 3 系统架构与实验设计

### 3.1 三层系统架构

| 层次     | 构成                                                 | 作用                   |
|--------|----------------------------------------------------|----------------------|
| AI 应用层 | llama.cpp + Qwen2.5 GGUF 模型                        | 模型推理与性能测试            |
| 目标运行环境 | QEMU RISC-V 模拟器 + openEuler 24.03 LTS-SP3 (RISC-V) | 提供 RISC-V Linux 运行环境 |
| 开发环境   | Ubuntu / WSL + riscv64 交叉编译工具链                     | 生成 riscv64 可执行文件     |

3.2 工作流程

① Ubuntu 交叉编译 → ② 部署传输 → ③ 运行推理  
生成 riscv64 可执行文件    拷贝至 RISC-V 系统    加载 GGUF 模型并测试

即“Ubuntu 交叉编译 → RISC-V 系统运行”，开发环境与运行环境分离。

3.3 两阶段实验方案

- 1. **Baseline 基线阶段**：-march=rv64gc\_zba\_zbb\_zbs，RVV 关闭，完成功能验证与初始性能测试；
- 2. **RVV 优化阶段**：改为 rv64gcv 并开启 GGML\_RVV=ON，重新编译复测，观察向量扩展对推理性能的影响。

4 实验环境搭建

| 组件     | 选型                                 |
|--------|------------------------------------|
| 宿主开发环境 | Ubuntu / WSL                       |
| 虚拟平台   | QEMU（RISC-V 模拟）                    |
| 操作系统   | openEuler 24.03 LTS-SP3（RISC-V 版本） |
| 交叉工具链  | riscv64-linux-gnu-gcc-14 / g++-14  |

选型理由：QEMU 模拟方案无需真实硬件即可开展 RISC-V 软件生态验证，是开发板缺失时的标准做法；openEuler 24.03 LTS-SP3 RISC-V 版本提供完整的 RISC-V Linux 运行环境。

环境确认：通过 /proc/cpuinfo 查看 CPU ISA 扩展，确认运行环境实际支持的指令集（实测显示含 v 扩展：isa: rv64imafdcvh），确保编译目标指令集与运行环境匹配——指令集不匹配会导致程序运行时直接报非法指令。

5 llama.cpp 移植（交叉编译）

5.1 编译配置要点

| 配置项   | Baseline 基线                       | RVV 优化版                           |
|-------|-----------------------------------|-----------------------------------|
| 工具链   | riscv64-linux-gnu-gcc-14 / g++-14 | riscv64-linux-gnu-gcc-14 / g++-14 |
| 目标平台  | CMAKE_SYSTEM_PROCESSOR=riscv64    | 同左                                |
| 指令集   | -march=rv64gc_zba_zbb_zbs         | -march=rv64gcv_zba_zbb_zbs（新增 v）  |
| ABI   | -mabi=lp64d                       | -mabi=lp64d                       |
| RVV 开 | GGML_RVV=OFF                      | GGML_RVV=ON                       |

关  
产物 llama-cli、llama-server 同左

编译成功后即得到可在 RISC-V 系统内直接运行的二进制。

5.2 配置解读

- rv64gc 提供 64 位通用指令集兼容性；
- zba / zbb / zbs 位操作扩展可供 GGML 推理内核使用；
- lp64d 确保浮点运算与 LP64 ABI 正确；
- 产物分工：llama-cli 命令行推理测试；llama-server 推理服务。

6 模型部署

| 模型                    | 参数量  | 量化     | 定位                  |
|-----------------------|------|--------|---------------------|
| Qwen2.5-0.5B-Instruct | 0.5B | Q4_K_M | 轻量级，适合资源受限场景，验证链路主力 |
| Qwen2.5-1.5B-Instruct | 1.5B | Q4_K_M | 能力更强，对算力要求更高，测试性能上限 |

选型目的：一个验证轻量可用，一个测试性能上限。两个模型均以 GGUF 量化格式成功加载。

7 功能验证

Baseline 阶段在 RISC-V 模拟环境中执行了多类功能测试，结果全部通过：

- 中文问答：能理解中文自然语言问题并给出回答；
- RISC-V 架构知识解释：能围绕指定主题给出解释性回答；
- C 语言代码生成：能按要求生成代码；
- 数学推理：能正确理解自然语言描述的数学问题、建立数学模型并完成方程求解。

功能测试表明模型不仅“能跑”，而且能真实完成推理任务，功能链路闭环。

8 Baseline 性能测试结果

8.1 性能数据

| 模型 | Prompt 速度 | Generation 速度 | 性能档位 |
|----|-----------|---------------|------|
|----|-----------|---------------|------|

|                       |             |             |       |
|-----------------------|-------------|-------------|-------|
| Qwen2.5-0.5B-Instruct | 3.5 token/s | 2.1 token/s | 轻量可用  |
| Qwen2.5-1.5B-Instruct | 0.6 token/s | 0.5 token/s | 模拟器受限 |

## 8.2 初步观察

- 模型参数量约为 3 倍（0.5B → 1.5B）时，推理速度出现数倍的明显下降，1.5B 在模拟器环境下受限于模拟开销；
- 0.5B 模型在纯软件模拟（QEMU）环境下仍能达到 **2 token/s** 以上的生成速度，对纯模拟环境而言已是“轻量可用”水平。

---

# 9 RVV 向量扩展优化实验

## 9.1 动机

Baseline 全部结果基于标量（RVV 关闭）编译。RISC-V 的向量扩展（RVV）是其在 AI / 并行计算场景下的重要特性，故在第二阶段开启 RVV 重新编译复测。

## 9.2 关键步骤

- 修改 riscv64-toolchain.cmake: -march=rv64gc → rv64gcv（新增 v 扩展）；
- 删除 build-riscv 旧缓存（避免 GGML\_RVV=OFF 残留）；
- cmake 重新配置时增加 -DGGML\_RVV=ON，确认输出出现 RVV 启用标志；
- j8 多核编译，用 file 命令校验产物确为 RISC-V ELF 可执行文件。

## 9.3 复测结果（Qwen2.5-0.5B）

| 指标            | Baseline    | RVV 优化版     |
|---------------|-------------|-------------|
| Prompt 速度     | 3.5 token/s | 3.3 token/s |
| Generation 速度 | 2.1 token/s | 1.8 token/s |

## 9.4 结果分析

RVV 优化版与基线**基本持平（略有下降）**。原因在于：QEMU 模拟器下 RVV 指令的**模拟开销较大**，向量化带来的收益被模拟成本抵消。因此，**RVV 的加速效果需要真实 RISC-V 硬件进一步验证**，这也构成本课题后续工作的首要方向。

---

# 10 讨论与局限性

- 模拟器 ≠ 真实硬件**：QEMU 是软件模拟，绝对性能与真实芯片存在数量级差异，结论只适用于“软件生态可行性”层面；
- RVV 结论待硬件验证**：向量扩展的收益在模拟器中无法充分体现，需在真实 RISC-V 开发板上复测；

- 3. **数据规模有限**：测试覆盖 2 个轻量模型、多类功能场景，属单次多轮的观察数据，未做大样本统计；
- 4. **量化策略单一**：本次统一使用 Q4\_K\_M，未对比更高 / 更低 bit 量化对精度与速度的影响。

---

## 11 结论

本实验完成了从环境搭建、软件移植到模型推理测试的完整流程：

- 1. 搭建 QEMU RISC-V + openEuler 运行环境，完成 riscv64 交叉编译与 llama.cpp 移植；
- 2. 成功部署 Qwen2.5 系列 GGUF 量化模型，完成中文问答、知识解释、代码生成与数学推理测试；
- 3. 开启 RVV 向量扩展（rv64gcv + GGML\_RVV=ON）重新编译并复测，完成优化实验；
- 4. 验证了 RISC-V 架构运行轻量化大语言模型推理任务的能力。

代表性数字：**3.3 t/s**（RVV 版 Prompt）、**2.1 t/s**（Baseline 生成速度）、**2 个模型**（Qwen2.5 系列部署成功）。

**核心结论：RISC-V 平台运行轻量化大模型具有可行性。** 在纯模拟环境下轻量模型已具备可用性；一旦迁移到真实硬件并配合 RVV 向量扩展，性能仍有更大想象空间。

---

## 12 后续工作方向

- 1. **真实硬件验证 RVV**：在真实 RISC-V 开发板上复测，验证向量扩展的加速效果；
- 2. **线程级优化**：结合多线程 / NUMA 提升推理吞吐；
- 3. **更大规模模型**：测试 3B / 7B 等更大模型的部署能力；
- 4. **llama-server API 服务**：将推理能力以标准 API 形式对外提供。

---

## 附录 A：复现要点速览（示意）

以下为关键配置的示意性整理，实际以课题操作记录为准。

**riscv64 工具链文件关键行（示意）**

```
set(CMAKE_SYSTEM_PROCESSOR riscv64)
set(CMAKE_C_COMPILER riscv64-linux-gnu-gcc-14)
set(CMAKE_CXX_COMPILER riscv64-linux-gnu-g++-14)
set(CMAKE_C_FLAGS "-march=rv64gcv_zba_zbb_zbs -mabi=lp64d")
set(CMAKE_CXX_FLAGS "-march=rv64gcv_zba_zbb_zbs -mabi=lp64d")
```

### 配置与编译（示意）

```
cmake -B build-riscv -DCMAKE_TOOLCHAIN_FILE=riscv64-toolchain.cmake \  
      -DGGML_RVV=ON -DCMAKE_BUILD_TYPE=Release  
cmake --build build-riscv -j8  
file build-riscv/bin/llama-cli    # 校验为 RISC-V ELF
```

### 运行推理（示意）

```
./llama-cli -m Qwen2.5-0.5B-Instruct-Q4_K_M.gguf -p "请解释什么是 RISC-V"  
"
```

### 环境确认

```
cat /proc/cpuinfo    # 查看 isa 扩展，确认 rv64gc / rvv 等特性
```